# Chapter 6

## 동적 프로그래밍
## (Dynamic Programming)

# 목차

- 동적 프로그래밍과 분할정복법
- 이항계수 구하기
- 최단경로(All-Pairs Shortest Path)
- 외판원 문제(Traveling Salesman Problem: TSP)

# 1. 동적 프로그래밍과 분할정복법

- 분할정복법
  - 하향식 해결법
  - 나누어진 부분들 사이에 서로 상관관계가 없는 문제를 해결하는데 적합
  - 상관 관계가 있을 경우 비효율적일 수 있음

- 동적 프로그래밍
  - 상향식 해결법
  - 분할정복법과 마찬가지로 문제를 나눈 후에 나누어진 부분들을 먼저 해결
  - 해결된 부분의 결과를 저장한 다음, 후에 그 결과가 필요할 때마다 저장된 값을 이용

    ↔ 분할정복법(매번 계산)

# 예: n번째 Fibonacci 항 구하기

분할정복법

```
int fib(int n)
{
    if (n <= 1)
        return n;
    else
        return fib(n-1) + fib(n-2);
}
```

fib(10)을 호출할 경우, fib(8)이 2번 실행

동적 프로그래밍

```
int fib2(int n)
{
    index i;
    int f[0..n];

    if (n <= 1)     return n;
    f[0] = 0; f[1] = 1;
    for (i = 2; i <= n; i++)
            f[i] = f[i-1]+f[i-2];
    return f[n];
}
```

fib2(10)을 호출할 경우, fib2(8)의 결과는 f[8]에 이미 저장.

# 동적 프로그래밍 알고리즘의 개발절차

- 개요
  - 주어진 문제를 여러 개의 작은 문제로 분할
  - 작은 문제들의 해를 먼저 구하여 <u>저장</u>
  - 더 큰 문제의 해를 구할 때 작은 문제의 해를 사용
  - 핵심

    작은 문제의 해로부터 큰 문제의 해를 구하는 관계식을 설정

# 동적 프로그래밍의 특징과 장단점

- 특징
  - 주어진 문제의 해를 구하기 위한 재귀 관계식(Recursive Property)을 구성하여야 한다.
  - 상향식으로 작은 부분 문제부터 해를 구하여야 한다.
- 장점
  - 프로그램을 구현할 때에는 필요한 모든 가능성을 고려해서 구현하게 된다. 따라서 동적 프로그래밍을 이용하여 항상 최적의 결과를 얻을 수 있다.
- 단점
  - 모든 가능성에 대한 고려가 불충분할 경우 최적의 결과를 보장할 수 없다. 동적 계획법을 구현하기 위해서는 충분히 많은 가능성에 대한 고려를 해야 한다.
  - 다른 방법론에 비해 많은 표(배열)을 이용하므로 메모리를 많이 요구한다.

# 2. 동적 프로그래밍의 예

- 이항 계수 구하기
- 최대 합 구하기
  - 변형: 행렬 경로 문제

# 2.1 이항 계수 구하기

이항계수 구하는 공식

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \quad \text{for } 0 \le k \le n$$

Overflow가 쉽게 발생하는 $n!$이나 $k!$을 계산하지 않고
통상 다음 식을 사용한다.

$$\binom{n}{k} = \begin{cases} \binom{n-1}{k-1} + \binom{n-1}{k} & \text{if } 0 < k < n \\ 1 & \text{if } k = 0 \text{ or } k = n \end{cases}$$

$$\frac{(n-1)!}{(k-1)!\,(n-k)!} + \frac{(n-1)!}{k!\,(n-1-k)!}$$

$$= \frac{k(n-1)!}{k!\,(n-k)!} + \frac{(n-k)(n-1)!}{k!\,(n-k)!}$$

# 알고리즘: 분할정복법

- 문제: 이항계수를 계산한다.
- 입력: 음수가 아닌 정수 n과 k, 여기서 k ≤ n
- 출력: bin, 이항계수 $\binom{n}{k}$
- 알고리즘:

```
public int bin(int n, int k) {
        if (k == 0 || n == k)
                return 1;
        else
                return bin(n-1, k-1) + bin(n-1, k);
}
```

# 알고리즘: 분할정복법(계속)

- 분할정복 알고리즘은 작성하기는 간단하지만, 효율적이지 않다.
- 이유?
  - 알고리즘을 재귀호출(recursive call)할 때 같은 계산을 반복해서 수행
- 예를 들면, bin(n-1,k-1)과 bin(n-1,k)는 둘 다 bin(n-2,k-1)의 결과가 필요한데, 따로 중복 계산됨
- 기억할 것: 분할정복법은 원래 문제 크기와 거의 같은 크기를 가진 2개의 사례로 분할될 경우 항상 비효율적!

# 동적 프로그래밍 알고리즘 설계전략

1. 재귀 관계식(recursive property)을 정립:
   - 2차원 배열 B를 만들고, 각 B[i][j]에는 $\binom{i}{j}$ 값을 저장
   - B[i][j]를 계산하기 위한 식
     - B[i][j] = B[i-1][j-1] + B[i-1][j]  0 < j < i
     - B[i][j] = 1  j = 0 또는 j = i

# 동적 프로그래밍 알고리즘 설계전략

- $\binom{n}{k}$를 구하기 위해서는 아래와 같이 B[0][0]부터 시작하여 위에서 아래로 재귀 관계식을 적용하여 배열을 채워 나가면 된다. 결국 값은 B[n][k]에 저장된다.

| | 0 | 1 | 2 | 3 | 4 | | j | k |
|---|---|---|---|---|---|---|---|---|
| 0 | 1 | | | | | | | |
| 1 | 1 | 1 | | | | | | |
| 2 | 1 | 2 | 1 | | | | | |
| 3 | 1 | 3 | 3 | 1 | | | | |
| 4 | 1 | 4 | 6 | 4 | 1 | | | |
| | | | | | | B[i-1][j-1] | B[i-1][j] | |
| | | | | | | | B[i][j] | |
| i | | | | | | | | |
| n | | | | | | | | |

# 알고리즘: 동적 프로그래밍

- 문제: 이항계수를 계산한다.
- 입력: 음수가 아닌 정수 n과 k, 여기서 k ≤ n
- 출력: bin, 이항계수 $\binom{n}{k}$
- 알고리즘:

```
public int binDynamic(int n, int k) {
    int[][] B = new int[n+1][k+1];

    for (int i = 0; i <= n; i++)
        for (int j = 0; j <= Math.min(i, k); j++) {
            if (j == 0 || j == i)
                B[i][j] = 1;
            else
                B[i][j] = B[i-1][j-1] + B[i-1][j];
        }
    return B[n][k];
}
```

# 알고리즘의 분석

- 단위연산: for-j 루프 안의 문장
- 입력의 크기: n, k
- 총 수행횟수는:

| i의 값 | 0 | 1 | 2 | 3 | … | k | k+1 | … | n |
|---|---|---|---|---|---|---|---|---|---|
| for-j 루프의 수행횟수 | 1 | 2 | 3 | 4 | … | k+1 | k+1 | … | k+1 |

$$1 + 2 + 3 + \cdots + k + \underbrace{(k+1) + \cdots (k+1)}_{n-k+1 \text{ times}}$$

$$= \frac{k(k+1)}{2} + (n-k+1)(k+1) = \frac{(2n-k+\ )(k+1)}{2} \in \Theta(nk)$$

# 2.2 합이 최대인 부분 리스트 구하기

- 문제 설명
  - 입력: 실수 배열 A[n]에 저장된 리스트
    - 예: 2.3  3.2  -4.5  2.1  -5.3  3.6  4.1  -2.3  3.5  -4.5
  - 출력: 연속된 수들의 합이 최대인 리스트를 찾아, 그 합과 리스트를 출력
    - 예: 합 = 8.9, 리스트 = 3.6  4.1  -2.3  3.5

# 해결 방법

- 어떻게 풀 것인가?
  - A[i]에서의 최대 합을 B[i]에 저장하자.
  - B[i]를 계산하기 위해서는 B[i-1]이 무엇인지 알아야지.
    - B[i-1] > 0, 리스트 연장 (B[i] = B[i-1] + A[i])
    - B[i-1] <= 0, 리스트를 새로 시작 (B[i] = A[i])
  - i를 0부터 n까지 증가하는 반복문으로 구현
    - B[0] = A[0]로 초기화
  - 이렇게 계산된 B[] 중에서 제일 큰 값이 해

  - 리스트의 출력 방법?

# 알고리즘

```
int maxSubList(double[] A, int n) {
    double[] B, max;

    B = new double[n];
    max = B[0] = A[0];
    for (int i = 1; i < n; i++) {
        B[i] = (B[i-1] <= 0) ? A[i] : B[i-1] + A[i];
        if (max < B[i])
            max = B[i];
    }
    return max;
}
```

리스트도 출력할 수 있도록 수정해볼 것!

# 행렬 경로 문제

- 규칙
  - int A[n][n]행렬이 주어지고, A[0][0]에서 시작하여 A[n-1][n-1]까지 이동
  - 현 위치에서 오른쪽이나 아래쪽으로만 이동할 수 있으며, 왼쪽, 위쪽, 대각선 이동은 허용하지 않는다
- 목표
  - A[0][0]에서 A[n-1][n-1]까지 이동하되, 방문한 칸에 있는 수들을 더한 값이 최대가 되도록 경로 설정
- 출력
  - 최대 경로의 합
  - 경로 출력 방법도 고려해 볼 것! (option)

# 이동 방법의 예

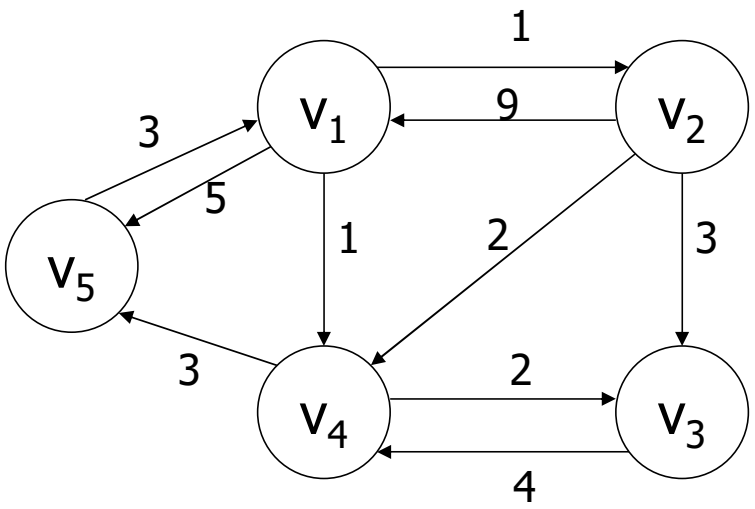
| | | | |
|---|---|---|---|
| 6 | 7 | 12 | 5 |
| 5 | 3 | 11 | 18 |
| 7 | 17 | 3 | 3 |
| 8 | 10 | 14 | 9 |

| | | | |
|---|---|---|---|
| 6 | 7 | 12 | 5 |
| 5 | 3 | 11 | 18 |
| 7 | 17 | 3 | 3 |
| 8 | 10 | 14 | 9 |

| | | | |
|---|---|---|---|
| 6 | 7 | 12 | 5 |
| 5 | 3 | 11 | 18 |
| 7 | 17 | 3 | 3 |
| 8 | 10 | 14 | 9 |

| | | | |
|---|---|---|---|
| 6 | 7 | 12 | 5 |
| 5 | 3 | 11 | 18 |
| 7 | 17 | 3 | 3 |
| 8 | 10 | 14 | 9 |

# 3. 최단 경로 구하기



# 최단경로 찾기(Shortest Path)

- 보기 : 한 정거장에서 다른 정거장으로 가장 빨리 갈 수 있는 버스 노선을 찾는 문제

- 문제: 가중치를 포함한 방향성 그래프에서 최단경로 찾기

- 최적화 문제(optimization problem)
  - 주어진 문제에 대하여 하나 이상의 많은 해답이 존재할 때, 이 가운데에서 가장 최적인 해답(optimal solution)을 찾아야 하는 문제
  - 최단경로 찾기 문제는 최적화 문제에 속한다.

# 무작정 알고리즘(brute-force algorithm)

- 한 정점에서 다른 정점으로의 모든 경로의 길이를 구한 뒤, 그들 중에서 최소 길이를 찾는다.

- 분석:
  - 그래프가 n개의 정점을 가지고 있고, 모든 정점들 사이에 에지가 존재한다고 가정
  - 한 정점 $v_i$에서 어떤 정점 $v_j$로 가는 경로들 중에서 나머지 모든 정점을 한번씩은 꼭 거쳐서 가는 경로들의 수를 우선 계산해 보자.
  - $v_i$에서 출발하여 처음에 도착할 수 있는 정점의 가지 수는 n-2개이고, 그 중에 하나를 선택하면, 그 다음에 도착할 수 있는 정점의 가지 수는 n-3개 이고, 이렇게 계속하여 계산해 보면, 총 경로의 개수는 (n-2)(n-3)…1 = (n-2)! 이 된다.
  - 이 경로의 개수 만 보아도 지수보다 훨씬 크므로, 이 알고리즘은 확실히 비효율적이다!

# 동적 프로그래밍: 자료구조

- 그래프의 인접행렬(adjacency matrix) 표현: W[][]
  - W[i][j] = weight  $v_i$에서 $v_j$로 가는 에지가 존재
  - W[i][j] = ∞  $v_i$에서 $v_j$로 가는 에지가 없음
  - W[i][j] = 0  i = j인 경우

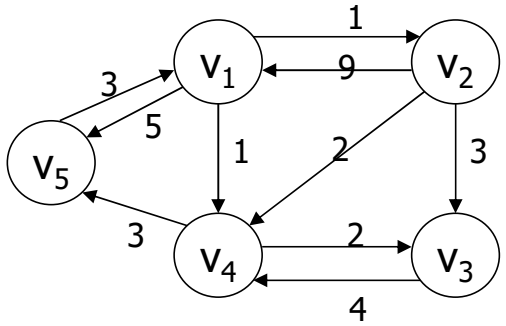
- 그래프에서 최단경로의 길이의 표현: D[][]
  - $D^{(k)}[i][j]$, $0 \le k \le n$
    - $\{v_1, v_2, \ldots, v_k\}$에 속하는 정점만을 중간 지점으로 사용하면서 $v_i$에서 $v_j$로 가는 최단 경로의 길이
    - **$W = D^{(0)}$**
    - 최종 결과 **$= D^{(n)}$**
  - **$D^{(0)}$를 가지고 $D^{(n)}$을 유도해낼 수 있는 방법**

# 동적 프로그래밍: 설계절차

- $D^{(k-1)}$을 가지고 $D^{(k)}$를 계산할 수 있는 재귀 계산식

$$D^{(k)}[i][j] = \text{minimum}(\underbrace{D^{(k-1)}[i][j]}_{\text{경우 1}}, \underbrace{D^{(k-1)}[i][k] + D^{(k-1)}[k][j]}_{\text{경우 2}})$$

  - 경우 1: $\{v_1, v_2, \ldots, v_k\}$의 정점들 만을 통해서 $v_i$에서 $v_j$로 가는 최단 경로가 $v_k$를 거치지 않는 경우
    - 예: $D^{(4)}[1][3] = D^{(5)}[1][3] = 3$
  - 경우 2: $\{v_1, v_2, \ldots, v_k\}$의 정점들 만을 통해서 $v_i$에서 $v_j$로 가는 최단 경로가 $v_k$를 거치는 경우
    - 예: $D^{(2)}[5][3] = D^{(1)}[5][2] + D^{(1)}[2][3] = 4+3=7$
    - 예: $D^{(2)}[5][4]$




- 상향식으로 k=1부터 n까지 이 과정을 반복

$$D^{(0)}, D^{(1)}, \ldots, D^{(n)}$$

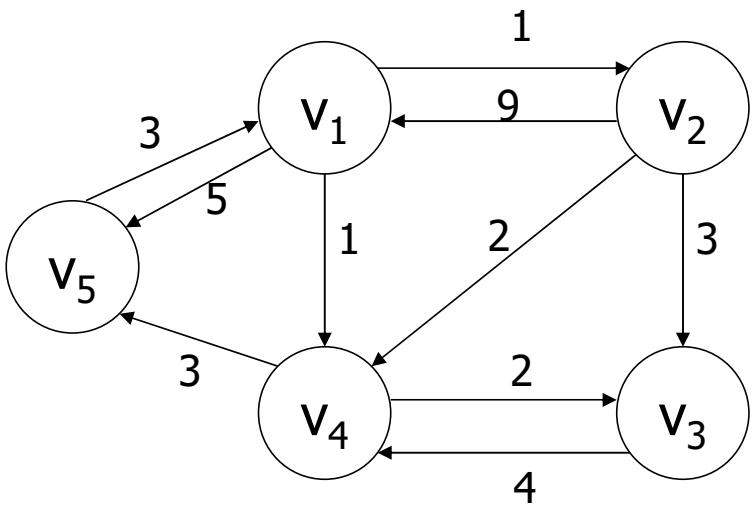
**Algorithm: Chapter 6 (Page 23)**

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | ∞ | 1 | 5 |
| 2 | 9 | 0 | 3 | 2 | ∞ |
| 3 | ∞ | ∞ | 0 | 4 | ∞ |
| 4 | ∞ | ∞ | 2 | 0 | 3 |
| 5 | 3 | ∞ | ∞ | ∞ | 0 |

**W**

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 3 | 1 | 4 |
| 2 | 8 | 0 | 3 | 2 | 5 |
| 3 | 10 | 11 | 0 | 4 | 7 |
| 4 | 6 | 7 | 2 | 0 | 3 |
| 5 | 3 | 4 | 6 | 4 | 0 |

**D**



# 동적 프로그래밍: 자료구조

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | ∞ | 1 | 5 |
| 2 | 9 | 0 | 3 | 2 | ∞ |
| 3 | ∞ | ∞ | 0 | 4 | ∞ |
| 4 | ∞ | ∞ | 2 | 0 | 3 |
| 5 | 3 | ∞ | ∞ | ∞ | 0 |

**W**

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 3 | 1 | 4 |
| 2 | 8 | 0 | 3 | 2 | 5 |
| 3 | 10 | 11 | 0 | 4 | 7 |
| 4 | 6 | 7 | 2 | 0 | 3 |
| 5 | 3 | 4 | 6 | 4 | 0 |

**D**

$D^{(0)}[2][5] = \infty$
$D^{(1)}[2][5] = 14$, $D^{(2)}[2][5] = 14$, $D^{(3)}[2][5] = 14$
$D^{(4)}[2][5] = 5$, $D^{(5)}[2][5] = 5$

# Floyd의 알고리즘 I – Pseudocode version

```
void floyd(int n, const number W[][], number D[][]) {
    int i, j, k;
    D = W;
    for (k=1; k <= n; k++)
      for (i=1; i <= n; i++)
        for (j=1; j <= n; j++)
                D[i][j] = minimum(D[i][j], D[i][k]+D[k][j]);
}
```

- 모든 경우를 고려한 분석:
  - 단위연산: for-j 루프 안의 지정문
  - 입력크기: 그래프에서의 정점의 수 n
- $T(n) = n \times n \times n = n^3 \in \Theta(n^3)$

# Floyd의 알고리즘 II

- 문제: 가중치 포함 그래프의 각 정점에서 다른 모든 정점까지의 최단거리를 계산하고, 각각의 최단경로를 구하라.
- 입력: 가중치 포함 방향성 그래프 W와 정점의 수 n.
- 출력: 최단경로의 길이가 포함된 배열 D, 그리고 다음을 만족하는 배열 P.
  - $v_i$에서 $v_j$까지 가는 최단경로의 중간에 놓여 있는 정점이 최소한 하나는 있는 경우 → 그 놓여 있는 정점 중에서 가장 큰 인덱스
  - 최단경로의 중간에 놓여 있는 정점이 없는 경우 → 0

# Floyd의 알고리즘 II – Java version

```
private static void floyd(int[][] W, int[][] D, int[][] P) {
        int i, j, k;
        for(i = 0; i < W.length; i++)        // vertex 번호는 0부터 시작한다고 가정
            for(j = 0; j < W.length; j++) {
                D[i][j] = W[i][j];           // W = D(0)
                P[i][j] = -1;                // 0번 vertex가 존재하므로, -1로 초기화
            }

        for(k = 0; k < W.length; k++)
            for(i = 0; i < W.length; i++)
                for(j = 0; j < W.length; j++)
                    if (D[i][k] + D[k][j] < D[i][j]) {
                        P[i][j] = k;
                        D[i][j] = D[i][k] + D[k][j];
                    }
}
```

# Floyd의 알고리즘 II: 예

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | ∞ | 1 | 5 |
| 2 | 9 | 0 | 3 | 2 | ∞ |
| 3 | ∞ | ∞ | 0 | 4 | ∞ |
| 4 | ∞ | ∞ | 2 | 0 | 3 |
| 5 | 3 | ∞ | ∞ | ∞ | 0 |

**W**

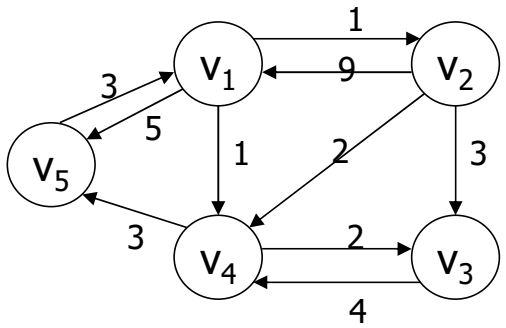
| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 3 | 1 | 4 |
| 2 | 8 | 0 | 3 | 2 | 5 |
| 3 | 10 | 11 | 0 | 4 | 7 |
| 4 | 6 | 7 | 2 | 0 | 3 |
| 5 | 3 | 4 | 6 | 4 | 0 |

**D**

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 4 | 0 | 4 |
| 2 | 5 | 0 | 0 | 0 | 4 |
| 3 | 5 | 5 | 0 | 0 | 4 |
| 4 | 5 | 5 | 0 | 0 | 0 |
| 5 | 0 | 1 | 4 | 1 | 0 |

**P**

이 예에서는 vertex 번호가 1부터 시작하므로, P는 0으로 초기화



```
D = W;
for(k=1; k<= n; k++)
    for(i=1; i <= n; i++)
        for(j=1; j<=n; j++)
            if (D[i][k] + D[k][j] < D[i][j]) {
                P[i][j] = k;
                D[i][j] = D[i][k] + D[k][j];
            }
```

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | ∞ | 1 | 5 |
| 2 | 9 | 0 | 3 | 2 | ∞ |
| 3 | ∞ | ∞ | 0 | 4 | ∞ |
| 4 | ∞ | ∞ | 2 | 0 | 3 |
| 5 | 3 | ∞ | ∞ | ∞ | 0 |

**W**

# 최단경로의 정점들 출력

- 알고리즘:

```
void path(index q, r) {
    if (P[q][r] != 0) {
            path(q, P[q][r]);
            System.out.println(" v" + P[q][r]);
            path(P[q][r], r);
    }
}
```

- 위의 P를 가지고 path(5,3)을 구해 보시오.

  path(5,3) = 4
  - path(5,4) = 1
    - path(5,1) = 0
    - v1
    - path(1,4) = 0
  - v4
  - path(4,3) = 0
  - 결과: v1 v4.
  - 즉, v5에서 v3으로 가는 최단경로는 v5, v1, v4, v3,이다.

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 4 | 0 | 4 |
| 2 | 5 | 0 | 0 | 0 | 4 |
| 3 | 5 | 5 | 0 | 0 | 4 |
| 4 | 5 | 5 | 0 | 0 | 0 |
| 5 | 0 | 1 | 4 | 1 | 0 |

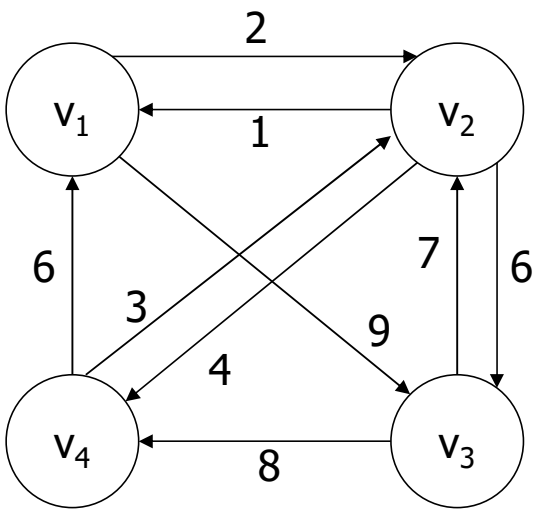
**P**

# 동적계획법에 의한 설계 절차

- 문제의 입력에 대해서 최적(optimal)의 해답을 주는 재귀 관계식(recursive property)을 설정
- 상향식으로 최적의 해답을 계산
- 상향식으로 최적의 해답을 구축

# 4. 외판원 문제(TSP)

- Tour (Hamiltonian circuit)
  - 한 정점에서 다른 모든 정점을 거쳐 다시 자기 자신으로 돌아오는 경로
  - optimal tour




length[$v_1$, $v_2$, $v_3$, $v_4$, $v_1$] = 22

length[$v_1$, $v_3$, $v_2$, $v_4$, $v_1$] = 26

length[$v_1$, $v_3$, $v_4$, $v_2$, $v_1$] = 21

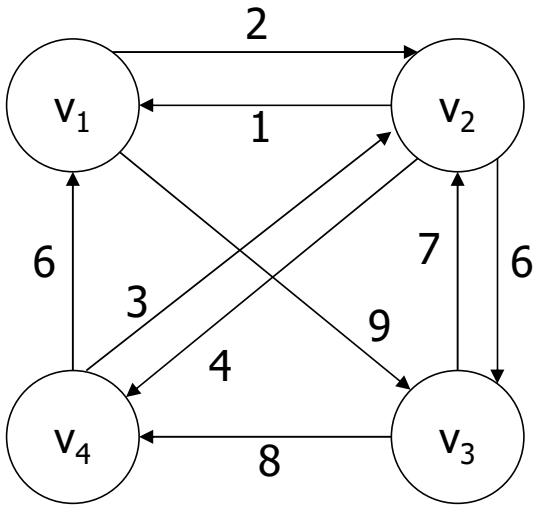
# 문제의 복잡성

- 각 정점마다 다른 정점과 n-1개의 연결이 존재한다고 가정할 경우, 정점 $v_1$에서 시작하는 tour의 수 =

  $(n-1)*(n-2)*\ldots*1 = (n-1)!$

- 동적 프로그래밍으로 해결 가능?
  - 가정: optimal tour = $v_1 \rightarrow v_k \rightarrow \ldots \rightarrow v_1$
  - Then, $v_k \rightarrow \ldots \rightarrow v_1$는 $v_k$에서 나머지 정점들을 한번만 거쳐서 $v_1$으로 가는 shortest path.
  - 따라서 최적의 원칙이 적용되며, 동적 프로그래밍을 적용 가능.

# TSP를 풀기 위한 자료구조




| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 2 | 9 | ∞ |
| 2 | 1 | 0 | 6 | 4 |
| 3 | ∞ | 7 | 0 | 8 |
| 4 | 6 | 3 | ∞ | 0 |

Adjacency Matrix: W

# TSP를 풀기 위한 동적 프로그래밍(1)

- V = 정점들의 집합,  A = V의 부분집합
- $D[v_i][A]$ = $v_i$에서 시작하여 A에 있는 정점들을 한번씩 지나면서 $v_1$까지 가는 최소 경로의 길이
  - A = ∅일 경우, $D[v_i][\varnothing] = W[i][1]$
  - A ≠ ∅일 경우,
    - $D[v_i][A] = \min(W[i][j] + D[v_j][A-\{v_j\}]),\ j{:}v_j \in A$.
  - A의 값을 ∅에서 V-{v1}까지 하나씩 증가.

- Example
  - $D[v_2][\varnothing] = 1,\ D[v_3][\varnothing] = \infty,\ D[v_4][\varnothing] = 6$
  - $D[v_2][\{v_3\}] = W[2][3] + D[v_3][\varnothing] = 6 + \infty = \infty$
  - $D[v_2][\{v_4\}] = W[2][4] + D[v_4][\varnothing] = 10$
  - $D[v_3][\{v_2\}] = 7 + 1 = 8,\ D[v_3][\{v_4\}] = 8 + 6 = 14$
  - $D[v_4][\{v_2\}] = 3 + 1 = 4,\ D[v_4][\{v_3\}] = \infty + \infty = \infty$

| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 2 | 9 | ∞ |
| 2 | 1 | 0 | 6 | 4 |
| 3 | ∞ | 7 | 0 | 8 |
| 4 | 6 | 3 | ∞ | 0 |

# TSP를 풀기 위한 동적 프로그래밍(2)

- A의 원소 수가 2개 일 경우
  - D[v2][{v3,v4}] = min(W[2][3] + D[v3][v4],
    W[2][4] + D[v4][v3])
    = min(6 + 14, 4 + ∞) = 20
  - D[v3][{v2,v4}] = min(7 + 10, 8 + 4) = 12
  - D[v4][{v2,v3}] = min(3 + ∞, ∞ + 8) = ∞

| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 2 | 9 | ∞ |
| 2 | 1 | 0 | 6 | 4 |
| 3 | ∞ | 7 | 0 | 8 |
| 4 | 6 | 3 | ∞ | 0 |

- A의 원소 수가 3개일 경우
  - D[v1][{v2,v3,v4}] = min(W[1][2] + D[2][{v3,v4}],
    W[1][3] + D[3][{v2,v4}],
    W[1][4] + D[4][{v2,v3}])
    = min(2+20, 9+12, ∞+∞) = 21

# 알고리즘

- 입력: 정점의 수 n, 그래프 W[][]
- 출력: optimal tour의 길이 minlength, 경로 P[][]

```
void travel(int n, number W[][], index P[][], number& minlength) {
    index i, j, k;
    number D[1..n][subset of V – {v1}];

    for (i = 2; i <= n; i++) D[i][∅] = W[i][1];
    for (k = 1; k <= n – 2; k++)
        for (all subsets of A ⊆ V – {v1} containing k vertices)
            for (i such that i ≠ 1 and vi is not in A) {
                D[i][A] = min(W[i][j] + D[j][A – {vj}]), j:vj ∈ A;
                P[i][A] = value of j that gave the minimum;
            }
    D[1][V – {v1}] = minimum(W[1][j] + D[j][A – {vj}]), 2 ≤ j ≤ n;
    P[1][V – {v1}] = value of j that gave the minimum;
    minlength = D[1][V – {v1}];
}
```

# 배열 P[][]를 이용한 최적 경로 출력

| 3 | 4 | 2 |
|---|---|---|
| P[1][{v2, v3, v4}] | P[3][{v2, v4}] | P[4][{v2}] |

- 첫 번째 노드의 인덱스 P[1][{v2,v3,v4}] = 3
- 두 번째 노드의 인덱스 P[3][{v2, v4}] = 4
- 세 번째 노드의 인덱스 P[4][{v2}] = 2
- 최적의 경로 = [v1, v3, v4, v2, v1]

# 복잡성 계산을 위한 사전 정리 3.1

$$\sum_{k=1}^{n} k\binom{n}{k} = \sum_{k=1}^{n} n\binom{n-1}{k-1}$$

$$= n\sum_{k=0}^{n-1}\binom{n-1}{k} = n\sum_{k=0}^{n-1}\binom{n-1}{k}1^k 1^{n-k}$$

$$= n(1+1)^{n-1} = n2^{n-1}$$

이항 정리(Binomial Theorem)

$$(a+b)^n = \sum_{k=0}^{n}\frac{n!}{k!(n-k)!}a^k b^{n-k}$$

# 시간 복잡성 계산

$$T(n) = \sum_{k=1}^{n-2} (n-1-k)k \binom{n-1}{k}$$

Note that $(n-1-k)\binom{n-1}{k} = (n-1)\binom{n-2}{k}$

위의 식에 대입하면

$$T(n) = (n-1)\sum_{k=1}^{n-2} k \binom{n-2}{k}$$

앞의 정리 3.1을 적용하면,

$$T(n) = (n-1)(n-2)2^{n-3} \in \Theta(n^2 2^n).$$

# 공간 복잡성

- D[][] 배열과 P[][] 배열의 크기?
  - number D[1..n][subset of V – {v1}]
  - V – {v1}의 부분집합의 수 = $2^{n-1}$
  - D[][] 배열의 크기 = $n*2^{n-1}$
  - P[][] 배열도 마찬가지.

- 공간 복잡도 $M(n) = 2 * n*2^{n-1} = n2^n \in \Theta(n2^n)$

# $(n-1)!$과 $(n-1)(n-2)2^{n-3}$의 차이점?

- n이 20이며, 단위 연산이 1μs일 경우
  - $19!\ \mu s = 3857$ years
  - $(20-1)(20-2)2^{20-3}\ \mu s = 45$ seconds